

MOSIP Decode 2026 -- Problem Statement 1

Automated Conformance Testing for Inji Certify and Inji Verify

Project Handbook

Shardul Chogale (conformance engine and evidence)

Mukta Varak (platform and delivery)

Written: 30 September 2026

Final submission deadline: 26 October 2026, 11:59 PM IST

Part 1 Mentor session brief -- Weekly Connect, 30 September

Part 2 Start to end: the clean guide to running the project

Part 3 Record of every step taken so far

Part 4 Planner, 30 September to 26 October, for both of us

This is the document to open first. It is kept current: when a procedure changes, Part 2 changes with it; when a day's work is done, Part 3 gains an entry and Part 4 gets its boxes ticked. The detailed evidence behind every claim lives in docs/findings/findings.tex.

Contents

Part 1	Mentor session brief	2
1.1	Where we are, in one paragraph	2
1.2	Three things to show if asked	2
1.3	Questions, ranked by how much the answer changes the build	2
1.4	What we want to leave with	4
1.5	Notes during the session	4
Part 2	Start to end: running the project from a clean machine	5
2.1	What runs, and why each piece is there	5
2.2	Step 1 -- Prerequisites	5
2.3	Step 2 -- The working directory	5
2.4	Step 3 -- ngrok, once	6
2.5	Step 4 -- Two local edits to Inji Verify	6
2.6	Step 5 -- Point the runner at your machine	6
2.7	Step 6 -- Start the stack, in this order	6
2.8	Step 7 -- Check everything is healthy	7
2.9	Step 8 -- Run the unit tests	7
2.10	Step 9 -- Run conformance	7
2.11	Step 10 -- Verify the results	7
2.12	Step 11 -- Record what the run showed	7
2.13	Step 12 -- Commit	8
2.14	Step 13 -- Shut down	8
2.15	Step 14 -- Ending the project	8
2.16	Troubleshooting -- problems that have already cost us time	8
Part 3	Record of every step taken	10
3.1	16-20 September -- registration and pre-kickoff groundwork	10
3.2	23 September -- official kickoff: plan, roles, contract	10
3.3	24 September -- the runner	10
3.4	29 September -- first live runs	10
3.5	30 September -- deadline found, this handbook	11
Part 4	Planner, 30 September to 26 October	12
4.1	Fixed points	12
4.2	Week 2 remainder -- Wed 30 Sep to Sun 4 Oct: finish the verifier side	12
4.3	Week 3 -- Mon 5 to Sun 11 Oct: the two halves meet	12
4.4	Week 4 -- Mon 12 to Sun 18 Oct: complete the mandatory set	13
4.5	Week 5 -- Mon 19 to Mon 26 Oct: submit	13
4.6	Dependencies between us	14
4.7	Risks, and what we do if they land	14

Part 1 Mentor session brief

Session MOSIP Decode Weekly Connect
 When Wednesday 30 September 2026, 5:00-6:00 PM IST
 Where <https://mosip.zoom.us/j/89100675963>
 Attending Shardul (and Mukta, if available)

1.1 Where we are, in one paragraph

We have a working harness that drives the OpenID Foundation conformance suite's REST API against Inji Verify with no human interaction. One command --

```
./run-conformance.sh -component verify --
```

creates the test plan, starts each module, obtains an authorisation request from Inji Verify's own API, delivers it to the suite, collects the result, and gates it against a recorded baseline. On 29 September its first unattended run reproduced our Day 1 manual run check for check: 15 passing checks, 4 failures, 3 warnings, the identical set of seven non-passing checks, and the same point of termination. Versions tested: Inji Verify 0.18.2, conformance suite 5.3.1. Inji Certify is not yet running.

1.2 Three things to show if asked

1. **The automated run matches the manual run exactly.** This is what makes everything else credible: the harness measures what a tester would see by hand.
2. **The nonce experiment.** Two runs differing only in where the nonce comes from. With the nonce the Verify UI's SDK sends, the entropy check warns (71.68 bits against 96 required) and the URL-safety check fails. When verify-service generates the nonce itself, both pass and nothing else changes. The weak nonce is `btoa(Date.now())` in the SDK; the service already does it correctly.
3. **The benchmark gate.** Inji Verify fails every module of the OID4VP 1.0 Final plan at DCQL extraction, so the gate fails a build on *change from the recorded baseline*, not on absolute pass rate. The suite's own verdict is never rewritten -- only the TestNG verdict is gated.

1.3 Questions, ranked by how much the answer changes the build

Each question has a default we will follow if there is no answer, so nothing blocks on the session. The top three are the ones worth the time.

#	Question	Why it matters	Our default if unanswered
1	Q1 -- Which spec version is the benchmark? Verify 0.18.2 implements Presentation Exchange; OID4VP 1.0 Final requires DCQL.	Decides whether any module can pass, and what the gate measures. Every 1.0 Final module currently stops at DCQL extraction.	1.0 Final as the primary benchmark (gated on regression); the ID2 plan run alongside to prove the pipeline reports passes. Both are config, so a later answer costs an edit.
2	Which Inji versions should we target? The problem statement says "Inji Certify and Inji Verify (on 1.0)" and "1.0.x" repositories. The newest images we could run are Verify 0.18.2 and Certify 0.14.0.	If "1.0" means a release line we are not running, every result we have is against the wrong version.	Stay on the latest published images and state the versions plainly in the submission.

#	Question	Why it matters	Our default if unanswered
3	Q7 -- Is there a lighter Inji Certify setup? docker-compose-injistack needs a PKCS12 keystore from Mimoto onboarding, a public DID endpoint, an external authorisation server and partner API keys.	Decides whether the combined run is achievable locally. Certify's issuer plan is 61 modules against the verifier plan's 12.	Attempt the local stack; if blocked, submit Verify end to end plus a documented Certify path, and say so plainly.
4	Where should findings go upstream? GitHub issues on mosip/inji-verify, or another tracker? Would a pull request fixing the SDK nonce be welcome from us?	This is our open-source contribution path. Our strongest finding is a three-line fix.	A GitHub issue on mosip/inji-verify, followed by a pull request from our fork.
5	Q4 -- Benchmark semantics. Regression-from-baseline, or an absolute target the build fails until conformance improves?	Changes what "pass" means in CI.	Regression-from-baseline (built). Absolute is one flag: -policy absolute-pass-rate.
6	Submission criteria. Is a demo video required? Must the api-testrig classes be merged upstream, or are branches on our forks acceptable?	Scopes the last week.	Assume a video is expected; deliver the Java classes as fork branches shaped as upstream pull requests.
7	Q2 -- HAIP. Verify hard-codes direct_post; HAIP requires direct_post.jwt. Track HAIP as a known-failing suite, or exclude it?	Affects scope, not architecture.	Exclude from the gate; document the gap.
8	Q3 -- Client identifier schemes. Verify offers did and pre_registered; the 1.0 Final plan offers redirect_uri, x509_san_dns, x509_hash. No overlap.	Explains one of the failures in every run.	Record as a known gap.
9	Q5 / Q8 -- Configuration questions. Is the shipped config.json meant to represent a certifiable deployment? Does anything rely on the presentation definition id, which the UI hard-codes?	Decides whether two observations become findings.	Keep both as observations, not findings.
--	Q6 -- answered by us; confirm only. Unattended runs work: the harness delivers the request by HTTP GET to the test's exposed authorization_endpoint, which is what the suite's paste box does internally.	Confirms the approach is acceptable.	Proceed.

1.4 What we want to leave with

At minimum: the version target (questions 1 and 2), the Certify route (question 3), and where to report findings (question 4). Everything else has a workable default.

1.5 Notes during the session

Q1	
Ver.	
Q7	
Issues	
Other	

After the session, record the answers in docs/findings/findings.tex (the Open Questions section) and update the planner in Part 4 where an answer changes it.

Part 2 Start to end: running the project from a clean machine

Follow this top to bottom on a machine that has never run the project. Every command has been run on the reference machine (macOS, Apple Silicon); Windows differences are noted where they exist. Budget two to three hours the first time, most of it image downloads.

2.1 What runs, and why each piece is there

Piece	Where it listens	Why it is needed
OpenID conformance suite 5.3.1	https://localhost.emobix.co.uk:8443	The examiner. It plays the wallet (verifier tests) or the client (issuer tests) and checks the component against the spec.
Inji Verify 0.18.2	UI :3000, API :8080 (path /v1/verify)	The verifier under test.
Inji Certify 0.14.0	not yet running	The issuer under test (Part 4, weeks 3-4).
ngrok tunnel	a *.ngrok-free.dev address	Mandatory. Verify identifies itself with a did:web DID resolved over public HTTPS, and bakes its public address into the response_uri the suite must reach from inside its own container.
The runner (this repo)	a command, not a server	Drives the suite's REST API and Verify's API, normalises results, applies the gate.

2.2 Step 1 -- Prerequisites

Git, Docker Desktop, Java 21, Maven 3.9+, Python 3.11+, ngrok. Check:

```
git --version; docker --version; java -version; mvn -version
python3 --version; ngrok version
```

macOS: `brew install git openjdk@21 maven python3 ngrok`, plus Docker Desktop from [docker.com](https://www.docker.com). Windows: winget for the same set, Docker Desktop with WSL 2, and once only: `git config --global core.autocrlf input` so shell scripts keep Unix line endings.

Java is 21, not 11: the Inji api-test projects require it, whatever older MOSIP documentation says.

2.3 Step 2 -- The working directory

The runner expects these six folders side by side under one root. The folder names matter: the runner finds Inji Verify's UI configuration at `../inji-verify`.

```
mkdir -p ~/projects/MOSIP && cd ~/projects/MOSIP
git clone https://github.com/shard-c6/mosip-decode-ps1.git
git clone --depth 1 https://github.com/mosip/inji-verify.git
git clone --depth 1 https://github.com/mosip/inji-certify.git
git clone --depth 1 https://github.com/mosip/mosip-functional-tests.git
git clone https://gitlab.com/openid/conformance-suite.git
git clone https://gitlab.com/openid/conformance-suite-automated-testing-tutorial.git
```

`mosip-decode-ps1` is ours. The Java work lands on branches of our forks of `inji-verify` and `inji-certify` (Shardul's forks at github.com/shard-c6; Mukta has write access to both).

2.4 Step 3 -- ngrok, once

Create a free ngrok account, then `ngrok config add-authtoken <token>` and claim the account's free static domain in the dashboard. Without a static domain the hostname changes on every restart and Step 4 must be redone each time. The domain is per-developer: Shardul's is `sphere-dust-sandlot.ngrok-free.dev`; Mukta needs her own.

2.5 Step 4 -- Two local edits to Inji Verify

These are not committed anywhere. A fresh clone lacks both, and either one missing produces failures that look like Inji Verify defects.

4a. In `inji-verify/docker-compose/docker-compose.yml` the literal placeholder `VERIFY_SERVICE_PROXY_FOR_LOCALHOST` appears in five environment variables and is substituted by nothing. Replace it with your domain -- hostname only, no `https://`, no trailing slash:

```
cd ~/projects/MOSIP/inji-verify/docker-compose
sed -i '' 's/VERIFY_SERVICE_PROXY_FOR_LOCALHOST/YOUR-DOMAIN.ngrok-free.dev/g' \
    docker-compose.yml # macOS; on Linux drop the ''
grep -c "YOUR-DOMAIN" docker-compose.yml # expect 5
```

Windows (PowerShell):

```
(Get-Content docker-compose.yml) -replace 'VERIFY_SERVICE_PROXY_FOR_LOCALHOST',
'YOUR-DOMAIN.ngrok-free.dev' | Set-Content docker-compose.yml
```

4b. In `inji-verify/docker-compose/config/config.json`, set the *Mock Identity (SD JWT)* entry to `"clientIdScheme":"pre_registered"`. It ships as `did`, which delivers the authorisation request by reference; the test variant needs it inline.

2.6 Step 5 -- Point the runner at your machine

`configs/runner.json` is shared and committed. Put anything that is yours in `configs/runner.local.json`, which git ignores and the runner deep-merges on top, matching components by name. Most setups need nothing here -- the runner talks to Verify on localhost. Example, to change the suite alias so two developers' plans don't collide:

```
{ "components": [ { "component": "inji-verify", "alias": "injiverify-mukta" } ] }
```

Install the runner's one dependency: `python3 -m pip install -r requirements.txt`.

2.7 Step 6 -- Start the stack, in this order

6a. Docker Desktop. Start it and wait for the engine: `docker info`.

6b. Conformance suite. Note the `-f`: without it, stop and start target the wrong file and report success while doing nothing.

```
cd ~/projects/MOSIP/conformance-suite
docker compose -f docker-compose-prebuilt.yml up -d
```

6c. Inji Verify.

```
cd ~/projects/MOSIP/inji-verify/docker-compose && docker compose up -d
```

6d. The tunnel -- leave it running in its own terminal:

```
ngrok http --url=https://YOUR-DOMAIN.ngrok-free.dev 3000
```

2.8 Step 7 -- Check everything is healthy

All three must answer before a run is worth attempting:

```
curl -k https://localhost.emobix.co.uk:8443/api/currentuser    # JSON, dev user
curl http://localhost:8080/v1/verify/did.json                # Verify, local
curl https://YOUR-DOMAIN.ngrok-free.dev/v1/verify/did.json   # Verify, public
```

The last is the one that matters most: if the public DID document does not resolve, nothing downstream can work. The path is `/v1/verify/did.json` -- there is no `.well-known`, because the DID has path segments.

2.9 Step 8 -- Run the unit tests

Needs no Docker. Normalisation is tested against the real Day 1 log, so these also check that the suite's field names have not drifted from our contract.

```
cd ~/projects/MOSIP/mosip-decode-ps1 && python3 -m pytest tests/ -q    # expect 50 passed
```

2.10 Step 9 -- Run conformance

```
./run-conformance.sh --component verify                    # the full verifier plan
./run-conformance.sh --component verify \
  --only oid4vp-1final-verifier-happy-flow -v             # one module, verbose
./run-conformance.sh --component verify --output scratch/run.json
./run-conformance.sh --component certify                  # once Certify is configured
./run-conformance.sh --combined                           # every configured component
```

Useful options: `-nonce-mode sdk|service` (Part 3, 29 September, for why this exists), `-policy absolute-pass-rate`, `-no-gate-exit` for exploratory runs only.

Exit status is the verdict, and CI keys on it:

```
0 gate passed -- nothing regressed against the baseline
1 gate failed -- a regression, or the harness could not execute a module
2 configuration error -- nothing was run
```

2.11 Step 10 -- Verify the results

A run that finished is not yet a run that is right. Check, in order:

1. The gate summary printed at the end. "Harness could not execute" means a fault in us or the environment, never a conformance result.
2. Each module's handoff. In the output JSON, verifier modules carry a handoff block: `delivered true`, `httpStatus 302`, and `secondsRemaining` well above zero. The request expires 300 s after issue.
3. Checks against the baseline. Every non-passing check carries a `findingRef` (F-01 ... F-09) when it matches a known finding. A failing check with no `findingRef` is new and needs reading before anything else.
4. The suite's own view at the `logUrl` for any module in doubt.
5. Archived logs under `logs/<date>/`. These are the JSON the runner fetched and are *unsigned*. For anything that will be reported upstream, export the signed log from the suite (*Download all Logs*) and commit the `.json` and `.sig` together.

2.12 Step 11 -- Record what the run showed

- A new or changed finding: a `\finding` block in `docs/findings/findings.tex`, tagged `CONFIRMED` or `UNVERIFIED`, with the log line quoted and the spec section the suite cited. Nothing goes upstream while `UNVERIFIED`.

- A day's work: a dated subsection in the findings Day Log -- objective, runs with test IDs, decisions with reasons, status.
- A module whose result is now understood: an entry in `configs/contract/expected-failures.json` with its reason and an evidence path. An expected failure without a reason is a silenced test.
- Every run worth keeping: a row in `logs/README.md`.

Rebuild the findings PDF:

```
cd docs/findings && pdflatex findings.tex && pdflatex findings.tex .
```

2.13 Step 12 -- Commit

Stage by path, never blindly: `git status -short -untracked-files=all` first. Subject in the imperative; body explaining why. No assistant attribution lines of any kind -- commits are made under our own names. Push to master on `shard-c6/mosip-decode-ps1`.

2.14 Step 13 -- Shut down

Leaving the stack running costs memory and is the commonest cause of "it worked yesterday".

```
cd ~/projects/MOSIP/inji-verify/docker-compose && docker compose down
cd ~/projects/MOSIP/conformance-suite && docker compose -f docker-compose-prebuilt.yml down
docker ps                                     # expect headers only
```

Then stop ngrok (Ctrl-C in its terminal). `down` keeps volumes, so the suite's plans and logs survive. Never add `-v` to the suite's `down` unless you mean to delete every plan and log it holds.

2.15 Step 14 -- Ending the project

The final week, in order. Detail and dates in Part 4.

1. Fresh-clone test: follow only this Part 2 on a machine that has never run the project. Anything missing gets fixed here, not explained later.
2. Final combined run, recorded; signed logs exported and committed.
3. Upstream: file confirmed findings as issues against `mosip/inji-verify`, and offer the fixes as pull requests from our forks.
4. Submission package: public repo with a clear README; the 1-2 page document or 5-7 slide pitch; demo video; the Docker Compose, test plan configuration, one-command runner and GitHub Actions workflow the problem statement lists.
5. Submit early (target 23 October). Multiple submissions are allowed and only the last before 26 October, 11:59 PM IST counts, so an early one is a safety net.
6. Hand-over: MOSIP may request the code be contributed. Keep the repository clean enough that this is a formality.
7. Shut the stack down (Step 13).

2.16 Troubleshooting -- problems that have already cost us time

Symptom	Cause and fix
Verify returns HTTP 400 to <code>vp-session-request</code>	The presentation definition has no id. The runner builds the UI's envelope; if this appears, check the credential name in <code>runner.json</code> matches <code>config.json</code> exactly.
A module fails at <code>request_uri</code>	The credential uses <code>did</code> (by reference). Redo Step 4b.

Symptom	Cause and fix
Failure that looks like an expired request	Requests die 300 s after issue. The runner refuses to deliver with under 30 s left; manually, check liveness with <code>curl .../v1/verify/vp-request/<id> first</code> .
Verify behaves as if unconfigured	Environment variables are read at container <i>creation</i> . After editing <code>docker-compose.yml</code> : down, then up -d. restart is not enough.
Suite "stopped" but still running	A plain docker compose down in that folder targets the wrong file. Always <code>-f docker-compose-prebuilt.yml</code> .
DID document 404	The path is <code>/v1/verify/did.json</code> ; no <code>.well-known</code> .
Gate passed but nothing ran	Fixed on 29 September: a harness error now fails the gate. If you ever see this again, it is a regression in the gate.
Unit tests pass, CLI crashes on import	Fixed on 29 September by import tests. Run the full <code>pytest tests/</code> , not a subset.
Browser warns the suite is not secure	Expected: self-signed certificate on a hostname that resolves to 127.0.0.1. Applies only because it is your own machine.

Part 3 Record of every step taken

What we did, in order, and what each step produced. Detail, evidence and quoted logs for every item are in docs/findings/findings.tex; this is the index to it.

3.1 16-20 September -- registration and pre-kickoff groundwork

1. Registered as a team for MOSIP Decode 2026 (confirmed 17 September) and chose Problem Statement 1.
2. Cloned the five upstream repositories side by side under /projects/MOSIP.
3. Started the conformance suite 5.3.1 locally from prebuilt images. Established that its local profile needs no API token.
4. Started Inji Verify 0.18.2 (three containers). Found the unsubstituted VERIFY_SERVICE_PROXY_FOR_LOCALHOST placeholder and replaced it with an ngrok hostname; found that did:web makes a public tunnel mandatory.
5. Created the verifier test plan 1N7C4DH1HvYmc (OID4VP 1.0 Final alpha, sd_jwt_vc / redirect_uri / url_query / plain_vp / direct_post).
6. Ran the happy-flow module by hand three times. Two runs were discarded, each for a reason we could prove: an expired request, then a variant mismatch fixed by switching the SD-JWT credential to pre_registered. Both had looked like Inji Verify defects. The third, MtzpWoD2dVgtic0, is the baseline.
7. Catalogued nine findings (F-01 to F-09), eight confirmed. Headline: Verify implements Presentation Exchange, not DCQL, so no 1.0 Final module can pass.
8. Wrote build-oid4vp-uri.py, the first piece of tooling; scaffolded the repository with setup, runbook, plan and findings documents.

3.2 23 September -- official kickoff: plan, roles, contract

1. Re-anchored the plan: the 20 September work predated the event.
2. Agreed roles by skill rather than by language. Mukta: platform and delivery. Shardul: conformance engine and evidence. Estimated split 49/51.
3. Chose to put the api-testrig work on our forks of inji-verify and inji-certify, shaped as upstream pull requests. api-test lives in those repositories, not in mosip-functional-tests.
4. Wrote the runner-to-testrig contract (docs/CONTRACT.md) with a fixture generated from the real Day 1 log, so the Java side need not wait for the runner.
5. Traced F-01/F-02 to btoa(Date.now()) in inji-verify-sdk, and found that verify-service already generates a sound nonce but only as a fallback. Drafted the upstream issue.

3.3 24 September -- the runner

1. Resolved the week's blocking unknown from the suite's source: a verifier module receives its authorisation request by an ordinary HTTP GET to the authorization_endpoint it exposes. No browser is needed.
2. Found the suite's built-in path for the screenshot requirement: a browser automation entry against an HTML stand-in page the suite serves itself.
3. Built the runner: configuration, suite client, Verify client, handoff, normalisation, gate, orchestrator, CLI, and run-conformance.sh.
4. 24 unit tests; one caught a defect in the gate before any live run.

3.4 29 September -- first live runs

1. Found and fixed two request defects before running, by reading Verify's DT0: no presentation definition was sent, and no nonce -- which would have made F-01 and F-02 silently vanish from automated runs.
2. Run 1 failed with HTTP 400: the definition lacked an id. Traced to the envelope the UI builds in vpVerificationState.ts; the runner now builds the same one.

3. Run 1 also exposed the most serious defect: the gate reported PASSED having measured nothing. Harness errors now always fail the gate.
4. Run 2, hcuWEvxGbvRgt7e: first fully unattended run. Matches the manual baseline check for check.
5. Run 3, Q9MaDZyfpJhjiDp: controlled experiment on the nonce source. Confirmed the F-01/F-02 root cause.
6. Emailed Mukta the directory setup and project status. 50 unit tests.

3.5 30 September -- deadline found, this handbook

1. Found the real deadline on the official event page: 26 October 2026, 11:59 PM IST, not the 21 October we had assumed. The plan gains five days.
2. Found the Weekly Connect schedule -- Wednesdays, 5:00-6:00 PM IST, from today.
3. Wrote this handbook: mentor brief, start-to-end guide, this record, the planner.

Part 4 Planner, 30 September to 26 October

S = Shardul (conformance engine and evidence). M = Mukta (platform and delivery). Tick boxes as work lands; move a slipped item forward rather than deleting it, so the planner stays an honest record.

4.1 Fixed points

Wed 30 Sep	Weekly Connect -- mentor brief, Part 1
Sun 4 Oct	End of week 2: verifier plan complete; api-test builds
Wed 7 Oct	Weekly Connect
Sun 11 Oct	End of week 3: runner output lands in the api-testrig report
Wed 14 Oct	Weekly Connect
Sun 18 Oct	End of week 4: minimum viable submission
Tue 20 Oct	Nothing new or risky starts after today
Wed 21 Oct	Weekly Connect -- show the working build if possible
Fri 23 Oct	First submission (safety net)
Mon 26 Oct	Final submission, 11:59 PM IST -- aim to finish by 6:00 PM

4.2 Week 2 remainder -- Wed 30 Sep to Sun 4 Oct: finish the verifier side

Day	S	M
Wed 30	☐ Weekly Connect; record answers in findings.tex ☐ Run modules 02-11 through the runner	☐ Directory setup (the 29 Sep email) ☐ Accept the two fork invitations ☐ Claim an ngrok static domain
Thu 1 Oct	☐ Triage modules 02-11: extend the baseline to all 12, each with a reason and evidence ☐ Export signed logs for any finding-bearing run	☐ Build apitest-commons from mosip-functional-tests ☐ api-test building and running in inji-verify
Fri 2 Oct	☐ Configure and run the ID2 verifier plan -- the likeliest source of a real pass ☐ Resolve F-05 from the UI's actual QR payload	☐ api-test building and running in inji-certify ☐ Write down the TestNG/Extent result shape
Sat 3 Oct	☐ Validate the screenshot mechanism on a module that gets past DCQL extraction ☐ Adjust the browser match pattern if needed	☐ Review CONTRACT.md; raise its four open points ☐ OpenIDConformanceTest skeleton against the fixture
Sun 4 Oct	☐ Joint sync (30 min): agree the Certify route from the mentor answer; confirm the week-3 split still holds. Buffer otherwise.	

Exit check: ./run-conformance.sh -component verify runs all 12 modules with a baseline entry for each; api-test builds in both repositories.

4.3 Week 3 -- Mon 5 to Sun 11 Oct: the two halves meet

Day	S	M
Mon 5	☐ Issuer-role support in the runner: no handoff; different plan configuration	☐ Unified Docker Compose: Verify and the suite in one project

Day	S	M
Tue 6	☐ Issuer plan configuration templates ☐ File the nonce issue upstream (channel per mentor answer)	☐ Compose: solve cross-project networking and the public hostname
Wed 7	☐ Weekly Connect ☐ Nonce fix as a pull request from the fork	☐ OpenIDConformanceTest in inji-verify's api-test, reading real runner output
Thu 8	☐ Combined mode: consolidated output across components	☐ Inji Certify environment: keystore, DID endpoint, auth server, keys -- day 1
Fri 9	☐ Signed exports; findings catch-up	☐ Certify environment -- day 2
Sat 10	☐ Architecture and automation documentation	☐ Certify environment -- day 3, or fall back (see risks)
Sun 11	☐ Joint integration session: runner output into the Extent report, end to end. Buffer otherwise.	

Exit check: a Verify run's results appear as TestNG pass/fail/skip in Inji Verify's own api-testrig report.

4.4 Week 4 -- Mon 12 to Sun 18 Oct: complete the mandatory set

Day	S	M
Mon 12	☐ Run the issuer plan against Certify	☐ Fold Certify into the unified compose
Tue 13	☐ Triage the issuer results; baseline for Certify	☐ OpenIDConformanceTest in inji-certify's api-test
Wed 14	☐ Weekly Connect ☐ Combined run end to end	☐ GitHub Actions workflow, first version
Thu 15	☐ Self-certification documentation	☐ Result diff between runs; consolidated report and badge
Fri 16	☐ Selective execution polish; expected-failures handling	☐ Actions: fail the build on a regression, demonstrably
Sat 17	☐ Findings written up as issue drafts	☐ Setup, troubleshooting and CI documentation
Sun 18	☐ Minimum viable submission check: all three invocations work and land in the api-testrig output.	

4.5 Week 5 -- Mon 19 to Mon 26 Oct: submit

Day	S	M
Mon 19	☐ Documentation complete; README final	☐ Fresh-clone test using only Part 2
Tue 20	☐ Fix whatever the fresh-clone test found	☐ Fix whatever the fresh-clone test found
Wed 21	☐ Weekly Connect: show the build; collect feedback	
Thu 22	☐ 1-2 page document or 5-7 slide pitch	☐ Demo video, end to end

Day	S	M
Fri 23	[] First submission.	
Sat 24-Sun 25	[] Final combined run recorded; signed logs [] File upstream issues	[] Final CI run green; video re-cut if needed
Mon 26	[] Final submission -- done by 6:00 PM IST; the deadline is 11:59 PM. Then shut down (Part 2, Step 13).	

4.6 Dependencies between us

This	needs	Blocked meanwhile?
OpenIDConformanceTest (M)	runner output (S)	No -- the committed fixture is real output shape
Issuer-plan runs (S)	Certify environment (M)	Yes, from week 4; issuer support is built in week 3 regardless
Combined run (both)	Certify environment and issuer support	Yes -- the reason Certify starts in week 3
GitHub Actions (M)	unified compose (M) and runner (S)	Partly -- the unit tests can run in CI from day one
Upstream issues (S)	mentor answer on channel	No -- default is a GitHub issue

4.7 Risks, and what we do if they land

Risk	Response
Certify cannot run locally	By Sat 10 Oct at the latest, fall back: submit Verify end to end, the issuer support in the runner, and a documented Certify path. Per-module conformance for one component satisfies the mandatory tasks.
Mukta's start slips	No reply to the 29 September email yet. The fixture means the Java class can start the moment setup is done; if api-test is not building by Fri 2 Oct, pair on it that weekend.
Neither of us writes Java	The class is kept small and logic-free by design. Escalate by mid-week 3 rather than absorb it quietly.
Screenshot mechanism fails	Only affects modules that get past DCQL. Fallback: one documented manual upload, stated plainly rather than hidden.
Mentor answers change the target	Plan and variants are configuration; a version change costs an edit and a re-run, not a rewrite.